Vietnamese-German University

Faculty of Engineering

MODULE HANDBOOK

Master of Computer Science (MCS) - Major Artificial Intelligence

In accordance with the specific examination regulation for
the **Master of Computer Science (MCS) - Major Artificial Intelligence**

Published by Faculty of Engineering
In September 2026
Version 1.0 - aligned with MCS Program Curriculum Version 1.0

Table of Contents

List of abbreviations 3
1. Objective of the program 4
2. Intended learning outcomes of the program 4
3. Program structure and curriculum 5
4. Curriculum Mapping 10
5. Module Descriptions 13

List of abbreviations

AH	Academic hour
AI	Artificial Intelligence
CE	Compulsory elective module
CM	Core module
CP	Credit point
ECTS	European Credit Transfer and Accumulation System
EM	Elective module
HPC	High Performance Computing
IKQ	Interdisciplinary Key Qualification
ILO	Intended learning outcome
MCS	Master of Computer Science
NLP	Natural Language Processing
RL	Reinforcement Learning
RTS	Research and Transferable Skills module
SM	Specialisation module
TBD	To be determined
VGU	Vietnamese-German University

1. Objective of the program

The Master of Science (M.Sc.) in Computer Science (MCS) - Major Artificial Intelligence at the Vietnamese-German University (VGU) is designed to equip students with the advanced knowledge and skills required to excel in the rapidly advancing field of Artificial Intelligence. This degree program prepares graduates for leading careers in AI research, autonomous systems, and intelligent software development by offering a comprehensive, hands-on education in cutting-edge neural technologies and research innovation.

The program provides students with in-depth expertise across the core pillars of computer science and artificial intelligence, focusing on Mathematics of Computing, Advanced Algorithms, Deep Learning, High Performance Computing and Data Engineering. This foundation is complemented by extensive coverage of specialized fields such as Natural Language Processing, Foundation Models & Generative AI, and Computer Vision. In addition, topics such as Robotics & Physical AI ensure students are well-versed in both the virtual and physical AI paradigms.

A strong emphasis is placed on practical, hands-on learning through a research approach. Through project work in the core modules, in the two Specialisation Tracks and in the elective modules of the third semester, students gain experience in designing, analyzing and implementing advanced computational solutions. This ensures that graduates not only grasp complex mathematical concepts but are also adept at using modern frameworks and tools in real-world engineering contexts, particularly in deploying AI solutions for industrial automation and secure digital infrastructures.

In addition to technical expertise, the program fosters the development of professional and research skills vital to thriving in both high-tech industries and academia. Students engage in a dedicated Advanced Seminar and in the Interdisciplinary Key Qualification Scientific Writing and Research. Together these modules cultivate collaboration, critical evaluation of scientific literature, and the written and methodological skills the Master Thesis requires. Professional responsibility, research integrity and the legal and societal implications of AI are treated in the Advanced Seminar and the Master Thesis, and in depth in the elective module AI Law, Ethics and Policy.

As a program following German academic standards, this curriculum offers students a truly international experience. They benefit from exposure to diverse academic perspectives, industry connections, and state-of-the-art computing facilities. The combination of specialized AI knowledge, hands-on training in modern software technologies, and professional development equips graduates with the confidence and skills necessary to succeed in the competitive and evolving global tech landscape.

Program Duration: Four semesters (two years) full-time. The full 120 ECTS are delivered, supervised and examined by the Faculty of Engineering at the Vietnamese-German University; the program carries no study-abroad or partner-university requirement.

Degree Awarded: Master of Science (M.Sc.). The study program "Computer Science - Major Artificial Intelligence" is recorded in the Transcript of Records and in the Diploma Supplement.

2. Intended learning outcomes of the program

The intended learning outcomes of the program are stated across the four competence dimensions of the Qualifications Framework for German Higher Education Degrees at master level: knowledge and understanding; use, application and generation of knowledge; communication and cooperation; and scientific self-conception and professionalism. They are attainable by every graduate and do not depend on

the choice of elective modules. The same wording is reproduced in Article 2(3) of the Specific Examination Regulation.

No.	Intended Learning Outcomes
ILO1	Knowledge and understanding: explain and critically discuss advanced concepts and theories in the mathematical, algorithmic, learning-theoretic, high-performance-computing and data foundations of computer science.
ILO2	Knowledge and understanding: explain the principles, the capabilities and the limitations of contemporary artificial-intelligence methods, including statistical and deep learning, language and generative models, and machine perception and robotics.
ILO3	Use and application of knowledge: design, implement and evaluate machine-learning and artificial-intelligence systems for complex problems, selecting methods appropriate to the data, the objective and the computational resources available.
ILO4	Use and application of knowledge: design and operate data and computing infrastructures that support artificial-intelligence workloads at scale, and reason about their performance, reliability and cost.
ILO5	Generation of knowledge: plan and conduct an independent piece of research - identify a research gap in the scientific literature, formulate a testable question, design and execute an empirical or theoretical investigation, and evaluate the outcome against established baselines.
ILO6	Generation of knowledge: interpret and validate empirical results using appropriate statistical methods and performance metrics, and state the limitations, threats to validity and uncertainty of the conclusions drawn.
ILO7	Communication: communicate technical work in writing and orally, in English, to specialist and non-specialist audiences and to the standards of scientific publication and professional practice.
ILO8	Cooperation: work effectively in multidisciplinary teams and carry a technical project through to delivery, including planning, documentation, version control and the reproducibility of results.
ILO9	Professionalism: assess the legal, ethical, societal and environmental implications of artificial-intelligence systems, and act in accordance with professional responsibility, research integrity, data protection and the rules governing the use of generative AI tools.
ILO10	Scientific self-conception: evaluate their own competence against the state of the art, and identify, plan and document the further learning required for professional practice and for doctoral-level work.

3. Program structure and curriculum

The Master of Science in Computer Science (MCS) - Major Artificial Intelligence program comprises 120 ECTS credit points, completed over four semesters with a uniform load of 30 ECTS per semester. The curriculum provides a balance between core foundational modules in Mathematics of Computing, Advanced Algorithms, Deep Learning, High Performance Computing and Data Engineering, and advanced study in Natural Language Processing, Foundation Models & Generative AI, Computer Vision & Perceptive AI and Robotics & Physical AI.

ECTS Credit Points	Credit Points	Semester			
		1	2	3	4
Core Module	30	18	12	0	0
Specialisation Module	24	12	12	0	0
Elective Module	30	0	0	30	0
Research and Transferable Skills Module	6	0	6	0	0
Master Thesis	30	0	0	0	30
Total ECTS Credit Points	**120**	**30**	**30**	**30**	**30**

Module Flexibility

The Core block and the Specialisation catalogue are taken in full: five core modules (30 ECTS) and the four modules of the two Specialisation Tracks (24 ECTS). The Elective catalogue offers eight modules (48 ECTS) of which 30 ECTS are earned in five slots in the third semester. The semester placement set out in the table above is binding; deviations require the approval of the examination board. The credit volume required in each module category is:

- **30 ECTS for Core Modules**
- **24 ECTS for Specialisation Modules**
- **30 ECTS for Elective Modules**
- **6 ECTS for Research and Transferable Skills Modules (Advanced Seminar and Interdisciplinary Key Qualification)**
- **30 ECTS** for Master Thesis

Core Modules (30 ECTS)

Core modules provide essential knowledge in Mathematics of Computing, Advanced Algorithms, Deep Learning, High Performance Computing and Data Engineering. These modules address advanced theoretical and practical topics that form the academic bedrock of the program. The five core modules are numbered in the order in which they are delivered: Core 1, Core 2 and Core 3 in the first semester, Core 4 and Core 5 in the second, so that the mathematical, algorithmic and learning-theoretic foundations are in place before the systems and data foundations, and all five before the Specialisation Tracks conclude.

Specialisation Modules (24 ECTS)

Specialisation modules allow students to deepen their expertise in specific areas of Artificial Intelligence. The catalogue comprises four modules (24 ECTS): Natural Language Processing, Foundation Models & Generative AI, Computer Vision & Perceptive AI, and Robotics & Physical AI. They form the two Specialisation Tracks and are all taken, in semesters 1 and 2.

Specialisation Tracks (12 ECTS each)

A Specialisation Track is a coherent pair of specialisation modules carrying 12 ECTS, taken with one module in the first semester and the second in the semester that follows. Both tracks are

compulsory. Each of the four specialisation modules is examined individually in accordance with its own module description; the track itself carries no separate concluding examination. The track is a teaching sequence and not a student choice: no separate track designation is awarded on the degree, the Transcript of Records or the Diploma Supplement.

Track 1 - Language & Generative AI: Specialisation 1 Natural Language Processing (semester 1) and Specialisation 2 Foundation Models & Generative AI (semester 2).

Track 2 - Perception & Robotics: Specialisation 3 Computer Vision & Perceptive AI (semester 1) and Specialisation 4 Robotics & Physical AI (semester 2).

The Specialisation catalogue consists of exactly these four modules, so the whole catalogue is earned and no specialisation credit is left unused.

A student therefore takes both Specialisation Tracks (24 ECTS) across semesters 1 and 2.

Elective Modules (30 ECTS)

Elective modules broaden the student's technical, systems and regulatory horizon. The catalogue comprises eight modules (48 ECTS) in four groups: Reinforcement Learning and Multi-Agent Systems & Agentic AI (Group 1, advanced AI methods); Distributed Systems and MLOps and AI Deployment (Group 2, systems and infrastructure); Security & Privacy and AI Law, Ethics and Policy (Group 3, trustworthy and responsible AI); and Mathematical Aspects of Differential Equations and Numerical Mathematics and Machine Learning Techniques for Engineering Applications (free choice). Students earn 30 ECTS in five elective slots in the third semester and must take at least one module from each of Groups 1 to 3, choosing from the modules announced for that academic year in accordance with Article 6(2) and Article 6(5) of the Specific Examination Regulation.

Research and Transferable Skills Modules (6 ECTS)

The Research and Transferable Skills block comprises the Advanced Seminar (3 ECTS) and the Interdisciplinary Key Qualification Scientific Writing and Research (3 ECTS), both taken in the second semester and both compulsory. Together with the Master Thesis these modules form the research progression of the program: in the Advanced Seminar the student learns to read and critique the state of the art and produces the exposé of the intended thesis topic; in the Interdisciplinary Key Qualification the student acquires the scientific writing, documentation and research-method skills the thesis requires; and in the Master Thesis of the fourth semester the student conducts independent research at full scale. Research-bearing credit therefore amounts to 36 ECTS, or 30 per cent of the degree.

The Advanced Seminar and the Interdisciplinary Key Qualification are delimited against each other: the Advanced Seminar addresses the reading, critical appraisal and oral presentation of the research literature and produces the thesis exposé, and the Interdisciplinary Key Qualification addresses scientific writing, documentation and research method. Both modules carry 3 ECTS. The volume below the customary minimum of 5 ECTS is justified by their function as focused skills modules preparing the Master Thesis, rather than as free-standing subject modules; the Faculty will review the merger of the two modules into a single 6-ECTS module at the first annual program review.

Master Thesis (30 ECTS)

The Master Thesis, completed during the fourth semester over a processing period of six months, is a research-based project that allows students to demonstrate their mastery of a specialized topic within the field of Artificial Intelligence. It is assessed by two examiners and concluded by a mandatory presentation and colloquium. The thesis represents the culmination of the program's academic requirements and is supervised by faculty members, often in collaboration with industry partners to solve sophisticated engineering problems.

Flexibility and Curriculum Updates

The curriculum is designed to ensure academic rigor and relevance to current industry trends. Details regarding modules, including **Module Names, Credit Points, Academic Hours, and Learning Activities**, are outlined in the module handbook. To remain aligned with the latest developments, modules may undergo changes reflecting cutting-edge research and industrial practices in the global IT sector.

Type Credit Points Academic Hours	Legend C=Compulsory E=Elective CE=Compulsory Elective ECTS Credit Points Contact hours per semester	Type	Semester	Credit Points (CPs)	Academic Hours	Weight for GPA
Module				**90**	**900**	
COMPULSORY ELECTIVE MODULES • **Core Modules** • **Specialisation Modules**				54		
	Core 1: Mathematics of Computing	CE	1	6	60	6
	Core 2: Advanced Algorithms	CE	1	6	60	6
	Core 3: Deep Learning	CE	1	6	60	6
	Core 4: High Performance Computing	CE	2	6	60	6
	Core 5: Data Engineering	CE	2	6	60	6
	Specialisation 1: Natural Language Processing	CE	1	6	60	6
	Specialisation 3: Computer Vision & Perceptive AI	CE	1	6	60	6
	Specialisation 2: Foundation Models & Generative AI	CE	2	6	60	6
	Specialisation 4: Robotics & Physical AI	CE	2	6	60	6
ELECTIVE MODULES - five of eight modules are taken - 30 CPs earned of 48 CPs offered				30		
	Elective 1: Reinforcement Learning	E	3	6	60	6
	Elective 2: Multi-Agent Systems & Agentic AI	E	3	6	60	6
	Elective 3: Distributed Systems	E	3	6	60	6
	Elective 4: MLOps and AI Deployment	E	3	6	60	6
	Elective 5: Security & Privacy	E	3	6	60	6
	Elective 6: AI Law, Ethics and Policy	E	3	6	60	6
	Elective 7: Mathematical Aspects of Differential Equations and Numerical Mathematics	E	3	6	60	6
	Elective 8: Machine Learning Techniques for Engineering Applications	E	3	6	60	6
COMPULSORY MODULES - Research and Transferable Skills Modules and Master Thesis - 36 CPs				36		
	Advanced Seminar	C	2	3	30	3
	Interdisciplinary Key Qualification: Scientific Writing and Research	C	2	3	30	3
	Master Thesis (incl. presentation / colloquium)	C	4	30	30	30

Note on the table above: the coursework of the program comprises 90 ECTS credit points and 900 contact hours, being the sum of the contact hours of the modules actually taken. The weight for the grade point average is the number of ECTS credit points of the module, in accordance with Article 5(2) of the Specific Examination Regulation. Within the Master Thesis module the thesis report accounts for 80 per cent and the colloquium for 20 per cent of the module grade.

4. Curriculum Mapping

Note: the map records the role of each module in delivering each intended learning outcome of the program.

I = Introduced - R = Reinforced - A = Assessed, that is, the module contributes a graded assessment of that outcome. Every outcome is assessed in at least one module taken by every student.

Module ID	Module Title	ILO1	ILO2	ILO3	ILO4	ILO5	ILO6	ILO7	ILO8	ILO9	ILO10
COMPULSORY ELECTIVE MODULES											
	Core 1: Mathematics of Computing	A		I			I				
	Core 2: Advanced Algorithms	A		R			R		I		
	Core 3: Deep Learning	R	A	A			R				
	Core 4: High Performance Computing	R			A		R		I		
	Core 5: Data Engineering			R	A		R		A		
	Specialisation 1: Natural Language Processing		R	A			R		R		
	Specialisation 2: Foundation Models & Generative AI		A	R			R			R	
	Specialisation 3: Computer Vision & Perceptive AI		R	A			A				
	Specialisation 4: Robotics & Physical AI		A	R	R				R		
ELECTIVE MODULES											
	Elective 1: Reinforcement Learning		R	R			R				
	Elective 2: Multi-Agent Systems & Agentic AI		R	R					R		
	Elective 3: Distributed Systems	R			R				R		
	Elective 4: MLOps and AI Deployment			R	A		R				
	Elective 5: Security & Privacy				R					R	
	Elective 6: AI Law, Ethics and Policy							R		A	
COMPULSORY MODULES											
	Elective 7: Mathematical Aspects of Differential Equations and Numerical Mathematics	R		R			R				
	Elective 8: Machine Learning Techniques for Engineering Applications		R	R			R				
	Advanced Seminar					I		A			I
	Interdisciplinary Key Qualification: Scientific Writing and Research					I		A			R
	Master Thesis (incl. presentation / colloquium)					A	A	A		A	A

5. Module Descriptions

The MCS program is structured into three categories: Compulsory, Compulsory Elective and Elective. Core and Specialisation modules are classified as Compulsory Elective in accordance with Annex A of the Specific Examination Regulation; all of them are taken by every student. Elective modules broaden the technical, systems and regulatory horizon and are chosen from the catalogue announced for the academic year, subject to the group requirement of Section 3.

The Compulsory track comprises the Advanced Seminar, the Interdisciplinary Key Qualification and the Master Thesis, and is mandatory for every student. Classifications are marked in the "Classification" checkboxes. Modules are updated periodically to reflect the latest technological advancements.

Draft

MODULE DESCRIPTION

Module title	**Core 1: Mathematics of Computing**
Module code	
Study program	**Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Tran Thi Thu Huong	huong.ttt@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☒ Compulsory Elective ☐ Elective

Semester 1

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Undergraduate calculus, linear algebra and discrete mathematics, and programming in Python with numerical libraries such as NumPy and SciPy. The module develops the probabilistic and optimisation foundations required by the later modules of the program and does not assume prior study of them at graduate level.

Intended learning outcomes
On successful completion of this module the learner will be able to:

1. Critically analyze complex computational problems using advanced matrix decomposition techniques and spectral theory.
2. Design and formulate rigorous mathematical models for high-dimensional data optimization in machine learning contexts.
3. Evaluate the convergence and stability of iterative numerical methods used in large-scale artificial intelligence systems.

Draft

4. Synthesize probabilistic frameworks to quantify uncertainty and algorithmic bias in automated decision-making models.

Contents

No.	Topic
1.	Rigorous foundations of linear algebra, vector spaces, inner products, and matrix factorization paradigms (LU, QR, and Singular Value Decomposition).
2.	Advanced multivariate calculus, gradient vectors, Jacobian and Hessian matrices, and Taylor series approximations for high-dimensional functions.
3.	Matrix calculus and automatic differentiation: the chain rule in matrix form, forward and reverse mode, and the mathematical basis of backpropagation.
4.	Unconstrained and constrained optimization theories, including Lagrange multipliers, KKT conditions, convex optimization, and semi-definite programming.
5.	First-order and second-order numerical optimization algorithms, focusing on stochastic gradient descent (SGD), Adam, and Newton-Raphson methods.
6.	Numerical linear algebra at scale: conditioning and stability, iterative solvers, randomised and low-rank approximation methods.
7.	Foundations of measure theory, continuous probability spaces, joint distributions, marginalization, and conditional expectation principles.
8.	Statistical estimation frameworks, including Maximum Likelihood Estimation (MLE), Maximum A Posteriori (MAP), and properties of estimators.
9.	Concentration inequalities and elements of information theory: entropy, cross-entropy, Kullback-Leibler divergence and mutual information.
10.	Discrete mathematics, graph theory structures, combinatorics, and recurrence relations applied to computational complexity modeling.

Learning activities

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Weekly problem sets focusing on the implementation of mathematical theories in Python.
Group Work	
Online Activities	Accessing digital repositories, Jupyter notebooks, and recorded tutorial sessions.

Draft

Activities	Explanation
Self-study	Independent research and assignments accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this core module.
Lab or Workshop	Integrated practical sessions focusing on numerical stability and computational efficiency.
Field Work	Not applicable for this core module.

Mode of assessment
Assignments (20 %): four individual assignments of problem sets combining analytical derivations with Python implementations, approximately 8 to 10 hours of student work each, submitted through the learning management system and graded individually.
Group project (30 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.
Final examination (50 %): written examination of 90 to 120 minutes, closed book; no materials or references are permitted in the examination room.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final exam	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Strang, G. (2023). *Introduction to Linear Algebra*, 6th ed. Wellesley-Cambridge Press.	**Required**	
Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press.	**Required**	
Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.	Recommended	

Draft

Material	Type	Material Locations
Bishop, C. M., & Bishop, H. (2023). *Deep Learning: Foundations and Concepts*. Springer.	Recommended	
Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations*, 4th ed. Johns Hopkins University Press.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

Draft

MODULE DESCRIPTION

Module title	**Core 2: Advanced Algorithms**
Module code	
Study program	**Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Tran Thi Thu Huong	huong.ttt@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☒ Compulsory Elective ☐ Elective

Semester 1

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Students are expected to have a solid foundation in Discrete Mathematics, Data Structures, and proficiency in at least one high-level programming language (e.g., Python, C++, or Java). Knowledge of computational complexity (Big O notation) and basic graph theory is highly recommended to engage with advanced algorithmic proofs.

Intended learning outcomes
On successful completion of this module the learner will be able to:

1. Analyze the asymptotic complexity of sophisticated recursive and non-recursive algorithms using master theorems and amortized analysis.
2. Design efficient algorithmic solutions for NP-hard problems using approximation techniques, heuristics, and randomized approaches.
3. Evaluate the trade-offs between space and time complexity for advanced data structures in high-performance AI computing contexts.

Draft

4. Critically synthesize algorithmic strategies to solve multi-objective optimization problems in real-world resource-constrained environments.
5. Implement and empirically benchmark advanced algorithmic solutions using Python/C++ and present performance analysis in a reproducible experimental report.

Contents

No.	Topic
1.	Rigorous analysis of amortized complexity and advanced recurrences in recursive algorithm design.
2.	Advanced Dynamic Programming paradigms including bitmasking and optimization of optimal substructure problems.
3.	Probabilistic analysis and Randomized Algorithms for efficient searching, sorting, and selection tasks.
4.	Greedy strategies and Matroid theory applied to complex network flow and scheduling problems.
5.	String matching algorithms and suffix structures for large-scale textual data processing in AI.
6.	Computational Geometry foundations including convex hulls, Voronoi diagrams, and spatial partitioning techniques.
7.	Graph theory advancements: Strongly connected components, Eulerian circuits, and maximum flow-min cut theorems.
8.	Approximation algorithms for NP-complete problems with guaranteed performance ratios and heuristic evaluation.
9.	Parallel and Distributed algorithms designed for multi-core architectures and massive dataset synchronization.
10.	Online algorithms and competitive analysis for decision-making under uncertainty and streaming data.

Learning activities

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Weekly problem sets involving complex algorithmic proofs and implementation-heavy coding tasks.
Group Work	Collaborative "whiteboard sessions" to architect solutions for high-level optimization challenges.
Online Activities	Accessing digital repositories, coding judge platforms, and supplemental video lectures.
Self-study	Independent research and coding practice accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this core module.
Lab or Workshop	Integrated practical sessions focusing on benchmarking algorithm performance in Python/C++.
Field Work	Not applicable for this core module.

Mode of assessment
Assignments (20 %): four individual assignments combining algorithmic proofs with implementation and empirical benchmarking in Python or C++, approximately 8 to 10 hours of student work each, graded individually.
Group project (30 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.
Final examination (50 %): written examination of 90 to 120 minutes, closed book; no materials or references are permitted in the examination room.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final exam	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms*, 4th ed. MIT Press.	**Required**	
Kleinberg, J., & Tardos, É. (2005). *Algorithm Design*. Pearson.	**Required**	
Williamson, D. P., & Shmoys, D. B. (2011). *The Design of Approximation Algorithms*. Cambridge University Press.	Recommended	
Motwani, R., & Raghavan, P. (1995). *Randomized Algorithms*. Cambridge University Press.	Recommended	
Knuth, D. E. (2011). *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms*. Addison-Wesley.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

Draft

MODULE DESCRIPTION

Module title	**Core 3: Deep Learning**
Module code	
Study program	**Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Nguyen Tuan Cuong	cuong.nt2@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☒ Compulsory Elective ☐ Elective

Semester 1

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Linear algebra, multivariable calculus and elementary probability, and proficiency in Python. Core 1 Mathematics of Computing is taught in parallel in semester 1 and supplies the optimisation and probability foundations on which this module draws. No prior study of neural networks is assumed; students without such exposure are advised to complete the pre-semester bridging offer [TBD – to be confirmed by the Faculty].

- Strong foundational knowledge in multivariable calculus, linear algebra, and probability theory.
- Advanced proficiency in Python programming and familiarization with scientific computing libraries.
- Successful completion of a bachelor-level introductory course in Artificial Intelligence, Data Science, or an equivalent technical discipline.

Intended learning outcomes
On successful completion of this module the learner will be able to:

Draft

1. Analyse a learning problem in statistical terms, distinguishing the hypothesis class, the loss, the optimisation procedure and the sources of generalisation error.
2. Derive and implement the training of deep neural networks, including backpropagation, initialisation, normalisation and modern first-order optimisation methods.
3. Design and implement architectures appropriate to the data modality – fully connected, convolutional, recurrent and attention-based – and justify the choice against alternatives.
4. Evaluate learned models rigorously using held-out protocols, appropriate metrics, calibration measures and statistically defensible comparisons against baselines.
5. Diagnose and remedy failure modes of learning systems, including overfitting, distribution shift, class imbalance, data leakage and unstable optimisation.

Contents *(Outline of subject and topics explored)*

No.	Topic
1.	Statistical foundations of learning: supervised, unsupervised and self-supervised settings, empirical risk minimisation, loss functions, and the bias-variance decomposition.
2.	Generalisation: capacity, regularisation, cross-validation, model selection, and elements of statistical learning theory.
3.	Classical statistical learning methods as baselines: linear and logistic regression, regularised regression, kernel methods and support vector machines, tree ensembles.
4.	Unsupervised and representation learning: clustering, principal component analysis, and self-supervised pretext tasks.
5.	Feed-forward neural networks: the multilayer perceptron, universal approximation, backpropagation and automatic differentiation.
6.	Optimisation of deep networks: stochastic gradient descent, momentum, Adam, learning-rate schedules, initialisation, batch and layer normalisation, and the treatment of vanishing and exploding gradients.
7.	Convolutional networks for images and signals: convolution, pooling, residual connections, and standard architectural families.
8.	Sequence models: recurrent networks and their limitations, the attention mechanism, and the transformer encoder as a general sequence architecture.
9.	Regularisation and training practice in deep learning: dropout, weight decay, data augmentation, early stopping, transfer learning and fine-tuning.
10.	Evaluation and diagnosis: metrics for classification, regression and ranking; calibration and proper scoring rules; error analysis, distribution shift, data leakage and reproducible experimental protocol.

Learning activities

Draft

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Homework assignments will be given on a regular basis. All assignments must be completed and submitted on time to contribute to the final grade.
Group Work	Collaborative problem-solving sessions during lectures to analyze optimization techniques.
Online Activities	Accessing lecture notes and digital references provided
Self-study	Independent learning and assignments accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this core module.
Lab or Workshop	Integrated into the weekly lecture sessions (two units of 90 minutes each).
Field Work	Not applicable for this core module.

Mode of assessment
Assignments (20 %): four individual assignments implementing learning algorithms from first principles and training and evaluating neural networks on provided datasets, approximately 8 to 10 hours of student work each, graded individually.
Group project (30 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.
Final examination (50 %): written examination of 90 to 120 minutes, closed book; no materials or references are permitted in the examination room.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final exam	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Draft

Material	Type	Material Locations
Bishop, C. M., & Bishop, H. (2023). Deep Learning: Foundations and Concepts. Springer.	**Required**	
Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.	**Required**	
Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning, 2nd ed. Springer.	Recommended	
Murphy, K. P. (2022). Probabilistic Machine Learning: An Introduction. MIT Press.	Recommended	
Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Dive into Deep Learning. Cambridge University Press.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

Draft

MODULE DESCRIPTION

Module title	Core 4: High Performance Computing
Module code	
Study program	**Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Tran Huu Tam	tam.th@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☒ Compulsory Elective ☐ Elective

Semester 2

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Computer architecture, operating systems and concurrent programming, and proficiency in at least one systems programming language (C or C++) as well as in Python. Core 3 Deep Learning is taught in semester 1 and supplies the neural-network background required by the material on the training of large models.

Intended learning outcomes

On successful completion of this module the learner will be able to:

1. Analyse the performance of a computation on modern parallel hardware, and determine whether it is bound by arithmetic throughput, by the memory hierarchy or by communication.
2. Design and implement parallel programs for shared-memory, distributed-memory and accelerator architectures, and reason about their strong and weak scaling behaviour.

3. Optimise numerical kernels for cache hierarchies, vector units and graphics processors, and quantify the speed-up obtained against a measured baseline.
4. Implement and evaluate the distributed training of a deep neural network using data, model or pipeline parallelism, and account for the communication cost incurred.
5. Measure, model and improve the performance of a high-performance application using profiling, scaling analysis and the roofline model, and report the result reproducibly.

Contents

No.	Topic
1.	Parallel architectures and machine models: multicore and manycore processors, SIMD and vector units, cache coherence and the memory hierarchy, NUMA effects, and the machine models used to reason about them.
2.	Performance analysis foundations: Amdahl's and Gustafson's laws, strong and weak scaling, arithmetic intensity, the roofline model, and the design of reproducible performance experiments.
3.	Shared-memory programming: threads and tasks, OpenMP work sharing, tasking and synchronisation, false sharing, thread affinity and NUMA-aware allocation.
4.	Distributed-memory programming: the message-passing model, MPI point-to-point and collective operations, communicators, non-blocking communication, and communication-computation overlap.
5.	Accelerator computing: the GPU execution and memory model, kernel design and launch configuration, occupancy, coalesced access, shared memory, streams and unified memory.
6.	Numerical kernels and libraries at scale: dense and sparse linear algebra, stencil and n-body computations, and the BLAS, LAPACK and FFT interfaces together with their accelerated implementations.
7.	Cache-aware and cache-oblivious optimisation: blocking and tiling, loop transformations, vectorisation, data layout, and the measurement of the resulting improvement.
8.	Input, output and storage for high-performance workloads: parallel file systems, collective and asynchronous input and output, checkpointing, and data staging.
9.	Distributed training of deep neural networks: data, model, tensor and pipeline parallelism; collective communication libraries; mixed precision; gradient accumulation; and the communication cost of each strategy.
10.	Cluster operation and performance engineering in practice: batch schedulers and job scripts, profiling and tracing tools, energy and cost awareness, and the reporting of performance results.

Learning activities *(Learning and teaching methods have to be described (e.g. lectures, exercises, seminars, internships, project work, self-study)*

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Programming assignments on shared-memory, message-passing and accelerator parallelism, each submitted with a short performance report containing a scaling study.
Group Work	Collaborative project optimising and scaling a numerical or deep-learning workload on the faculty cluster and its GPU nodes.
Online Activities	Accessing the cluster documentation, profiling and tracing tools, recorded tutorials, and collaborative version control repositories.
Self-study	Independent research, academic paper reading and software development accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this core module.
Lab or Workshop	Integrated sessions on the faculty cluster and GPU nodes: OpenMP and MPI exercises, CUDA kernels, and a distributed training run.
Field Work	Not applicable for this core module.

Mode of assessment

Assignments (20 %): four individual assignments - two implementing and benchmarking shared-memory and message-passing kernels, and two implementing and optimising accelerator kernels - each submitted with a short performance report and graded individually.

Group project (30 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.

Final examination (50 %): written examination of 90 to 120 minutes, closed book; no materials or references are permitted in the examination room.

Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final exam	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Hager, G., & Wellein, G. (2010). Introduction to High Performance Computing for Scientists and Engineers. CRC Press.	**Required**	
Pacheco, P., & Malensek, M. (2021). An Introduction to Parallel Programming, 2nd ed. Morgan Kaufmann.	**Required**	
Hennessy, J. L., & Patterson, D. A. (2019). Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann.	Required	
Kirk, D. B., & Hwu, W. W. (2022). Programming Massively Parallel Processors, 4th ed. Morgan Kaufmann.	Recommended	
Ben-Nun, T., & Hoefler, T. (2019). Demystifying Parallel and Distributed Deep Learning. ACM Computing Surveys, 52(4).	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

MODULE DESCRIPTION

Module title	**Core 5: Data Engineering**
Module code	
Study program	**Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Tran Duc Khanh	khanh.td@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☒ Compulsory Elective ☐ Elective

Semester 2

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Database systems (SQL), operating systems and proficiency in at least one high-level programming language. Core 4 High Performance Computing is taught in parallel in semester 2 and supplies the parallel-computing background.

Intended learning outcomes

On successful completion of this module the learner will be able to:

1. Analyze and architect end-to-end data pipelines that support the ingestion, transformation, and storage of massive, heterogeneous datasets.
2. Design scalable distributed storage solutions using polyglot persistence strategies to meet diverse consistency and availability requirements.
3. Evaluate the trade-offs between batch and stream processing paradigms for real-time AI model serving and analytics.

4. Critically synthesize data governance frameworks to ensure data quality, lineage, and privacy compliance within automated engineering workflows.

Contents

No.	Topic
1.	Architectural foundations of data engineering: lifecycle, personas, and the evolving ecosystem of modern data stacks.
2.	Advanced data modeling techniques: relational, document, graph, and wide-column stores for specialized AI workloads.
3.	Data ingestion strategies: bulk loading, change data capture (CDC), and API-based extraction for disparate sources.
4.	Distributed computing principles: MapReduce, Spark, and parallel processing frameworks for high-throughput data transformations.
5.	Stream processing architectures: real-time event handling using Kafka and Flink for low-latency perceptive AI feedback.
6.	Data warehousing and Data Lakehouses: designing scalable analytical storage using Delta Lake and Iceberg formats.
7.	Orchestration and workflow management: automating complex dependency graphs using tools like Airflow or Prefect.
8.	Data quality and observability: implementing automated testing, monitoring, and lineage tracking for robust pipelines.
9.	Security and governance: architecting for data privacy, encryption, and role-based access control in distributed environments.
10.	Engineering for AI: optimizing feature stores and data versioning to support reproducible machine learning operations.

Learning activities *(Learning and teaching methods have to be described (e.g. lectures, exercises, seminars, internships, project work, self-study)*

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Periodic technical labs involving the implementation of ETL/ELT pipelines and database optimization tasks.
Group Work	Collaborative engineering project focusing on building a functional data platform for a specific AI use case.

Activities	Explanation
Online Activities	Accessing cloud environments (AWS/GCP), digital documentation, and collaborative code repositories (Git).
Self-study	Independent research, coding development, and technical documentation accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this specialization module.
Lab or Workshop	Integrated sessions using the faculty's distributed computing cluster and containerized data tools.
Field Work	Not applicable for this specialization module.

Mode of assessment

Assignments (20 %): four individual assignments building and testing ingestion, transformation and storage components on provided datasets, graded individually.

Group project (30 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning. The component is assessed against the planning and delivery of the project, the quality of the documentation, the use of version control and the reproducibility of the results as well as against the artefact itself, and is the graded assessment of program intended learning outcome ILO8.

Final examination (50 %): written examination of 90 to 120 minutes, closed book; no materials or references are permitted in the examination room.

Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final exam	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.	**Required**	

Material	Type	Material Locations
Reis, J., & Housley, M. (2022). *Fundamentals of Data Engineering*. O'Reilly Media.	**Required**	
Shapira, G., Palino, T., Sivaram, R., & Petty, K. (2021). Kafka: The Definitive Guide, 2nd ed. O'Reilly Media.	Recommended	
Burns, B. (2018). *Designing Distributed Systems*. O'Reilly Media.	Recommended	
Karau, H., & Warren, R. (2017). *High Performance Spark*, 1st ed. O'Reilly Media.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

MODULE DESCRIPTION

Module title	**Specialisation 1: Natural Language Processing**
Module code	
Study program	**Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Nguyen Tuan Cuong	cuong.nt2@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☒ Compulsory Elective ☐ Elective

Semester 1

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Core 3 Deep Learning, taught in parallel in semester 1, together with probability, linear algebra and advanced Python programming. Students without prior exposure to neural networks are advised to complete the pre-semester bridging offer [TBD – to be confirmed by the Faculty].

Intended learning outcomes

On successful completion of this module the learner will be able to:

1. Analyze linguistic structure at the morphological, syntactic, semantic, and discourse levels, and formalize it into computational representations suitable for machine learning.
2. Design and implement neural sequence models - from distributional embeddings and recurrent networks to transformer encoders and decoders - for classification, tagging, parsing, and generation tasks.
3. Evaluate natural language processing systems using appropriate intrinsic and extrinsic metrics, statistical significance testing, and systematic error analysis, including in multilingual and low-resource settings.

4. Critically assess bias, toxicity, privacy, and factuality risks in language technologies and apply mitigation strategies within deployed processing pipelines.

Contents

No.	Topic
1.	Language data and preprocessing: corpora, tokenization and subword segmentation (BPE, WordPiece, SentencePiece), normalization, and annotation standards.
2.	Statistical foundations of language modelling: n-gram models, smoothing, information-theoretic evaluation, and perplexity.
3.	Lexical semantics and distributional representations: word2vec, GloVe, fastText, contextual embeddings, and semantic similarity.
4.	Sequence labelling: part-of-speech tagging, named-entity recognition, hidden Markov models, conditional random fields, and neural taggers.
5.	Syntactic and semantic parsing: constituency and dependency grammars, transition-based and graph-based parsers, and semantic role labelling.
6.	Neural architectures for language: the encoder-decoder transformer, cross-attention, decoding strategies (greedy, beam, sampling, constrained decoding), and the pre-train-and-fine-tune paradigm for language tasks. The attention mechanism and the transformer encoder are introduced in Core 3 Deep Learning.
7.	Text classification, sentiment analysis, and information extraction, including relation extraction and event detection.
8.	Machine translation and text generation: decoding strategies, automatic evaluation (BLEU, chrF, COMET), and human evaluation protocols.
9.	Question answering, retrieval-augmented pipelines, and task-oriented dialogue systems.
10.	Multilingual and low-resource natural language processing, and the responsible deployment of language technology: bias, toxicity, privacy, and factuality.

Learning activities *(Learning and teaching methods have to be described (e.g. lectures, exercises, seminars, internships, project work, self-study)*

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Programming assignments implementing tokenizers, taggers, and transformer-based models in PyTorch.

Activities	Explanation
Group Work	A team project building and evaluating an end-to-end natural language processing system on a multilingual dataset.
Online Activities	Accessing digital corpora, model hubs, Jupyter notebooks, and recorded tutorial sessions.
Self-study	Independent reading of research papers and assignments accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this specialization module.
Lab or Workshop	Integrated laboratory sessions on corpus annotation, fine-tuning, and systematic error analysis.
Field Work	Not applicable for this specialization module.

Mode of assessment

Assignments (20 %): four individual assignments implementing and evaluating language-processing models on standard benchmark corpora, graded individually.

Group project (30 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.

Final examination (50 %): written examination of 90 to 120 minutes, closed book; no materials or references are permitted in the examination room.

Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final exam	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Jurafsky, D., & Martin, J. H. Speech and Language Processing, 3rd edition draft. Stanford University, freely available online.	**Required**	

Material	Type	Material Locations
Eisenstein, J. (2019). Introduction to Natural Language Processing. MIT Press.	**Required**	
Goldberg, Y. (2017). Neural Network Methods for Natural Language Processing. Morgan & Claypool.	Recommended	
Tunstall, L., von Werra, L., & Wolf, T. (2022). Natural Language Processing with Transformers, rev. ed. O'Reilly.	Recommended	
Koehn, P. (2020). Neural Machine Translation. Cambridge University Press.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

MODULE DESCRIPTION

Module title	**Specialisation 2: Foundation Models & Generative AI**
Module code	
Study program	**Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Nguyen Tuan Cuong	cuoing.nt2@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☒ Compulsory Elective ☐ Elective

Semester 2

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Core 3 Deep Learning and Specialisation 1 Natural Language Processing, together with proficiency in Python and a deep-learning framework such as PyTorch.

Intended learning outcomes

On successful completion of this module the learner will be able to:

1. Analyze the mathematical foundations and probabilistic frameworks of latent variable models and diffusion-based generative processes.
2. Design and implement state-of-the-art transformer architectures and large language models (LLMs) for complex sequence generation tasks.
3. Evaluate the quality, diversity, and fidelity of synthetic data generated by GANs and VAEs using rigorous quantitative metrics.
4. Critically synthesize ethical mitigation strategies regarding deepfakes, algorithmic bias, and copyright implications in the deployment of generative AI.

Contents

No.	Topic
1.	Scaling and the economics of pre-training: scaling laws for model and data size, compute budgets, and the practical consequences for architecture choice. The mechanics of backpropagation, normalisation and optimisation are established in Core 3 Deep Learning and are not repeated here.
2.	Probabilistic generative modeling: Variational Autoencoders (VAEs), evidence lower bounds, and latent space manifold learning.
3.	Generative Adversarial Networks (GANs): minimax games, Wasserstein loss, and stable training for high-fidelity synthesis.
4.	From the transformer encoder to generative architectures: encoder-decoder and decoder-only designs, positional encoding schemes, long-context mechanisms and efficient attention. The attention mechanism itself is introduced in Core 3 Deep Learning.
5.	Large Language Models (LLMs): pre-training strategies, instruction tuning, and the scaling laws of neural networks.
6.	Autoregressive models and Flow-based models: exact likelihood estimation and invertible transformations for data generation.
7.	Diffusion models: forward and reverse stochastic processes for high-quality image and audio generation.
8.	Self-supervised pre-training at scale: contrastive objectives, masked autoencoders and next-token prediction as pre-training regimes, and the transfer behaviour they produce. The principle of self-supervision is introduced in Core 3.
9.	Multi-modal generative AI: CLIP, Stable Diffusion, and cross-modal alignment between text, image, and video.
10.	Parameter-efficient adaptation and deployment: LoRA and adapter methods, quantisation, prompt engineering, and the detection and provenance of AI-generated content.

Learning activities

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.

Activities	Explanation
Individual Assignments	Periodic technical coding labs focusing on training generative models and implementing neural architectures.
Group Work	Collaborative research projects focusing on fine-tuning LLMs or building custom generative pipelines.
Online Activities	Accessing GPU clusters, digital research repositories (arXiv), and collaborative JupyterHub environments.
Self-study	Independent research, paper reading, and intensive model training sessions (120 AHs).
Internship	Not applicable for this specialization module.
Lab or Workshop	Integrated sessions using faculty GPU clusters for large-scale generative model training.
Field Work	Not applicable for this specialization module.

Mode of assessment

Assignments (30 %): four individual assignments training, fine-tuning and evaluating generative models on provided datasets and compute allocations, graded individually.
Group project (20 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.
Final examination (50 %): written examination of 90 to 120 minutes, closed book; no materials or references are permitted in the examination room.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	30%
Group project	20%
Final exam	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Foster, D. (2023). Generative Deep Learning, 2nd ed. O'Reilly Media.	**Required**	
Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.	**Required**	
Tunstall, L., von Werra, L., & Wolf, T. (2022). Natural Language Processing with Transformers, 1st ed. O'Reilly Media.	Recommended	
Murphy, K. P. (2023). Probabilistic Machine Learning: Advanced Topics. MIT Press.	Recommended	
Selected Research Papers (Transformers, GANs, Diffusion Models, LLMs).	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

MODULE DESCRIPTION

Module title	**Specialisation 3: Computer Vision & Perceptive AI**
Module code	
Study program	**Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Nguyen Tuan Cuong	cuong.nt2@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☒ Compulsory Elective ☐ Elective

Semester 1

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Core 3 Deep Learning, taught in parallel in semester 1, together with linear algebra, digital image processing and proficiency in Python. Students without prior exposure to neural networks are advised to complete the pre-semester bridging offer [TBD – to be confirmed by the Faculty].

Intended learning outcomes

On successful completion of this module the learner will be able to:

1. Analyze and mathematically model the fundamental geometry of image formation and camera calibration for 3D reconstruction tasks.
2. Design and implement robust computer vision pipelines utilizing state-of-the-art convolutional neural networks and transformer architectures for object detection.
3. Evaluate the performance and limitations of perceptive AI systems in real-world constraints such as varying illumination and occlusions.

4. Critically synthesize multi-modal sensory data to develop advanced scene understanding models for autonomous or robotic applications.

Contents

No.	Topic
1.	Fundamentals of image formation including camera models, lens distortion, and projective geometry principles.
2.	Advanced feature detection, description, and matching using invariant local descriptors and robust estimation.
3.	Deep learning for vision: Architecture design of modern CNNs and vision transformer backbones.
4.	Object detection and recognition frameworks including two-stage (R-CNN) and single-shot (YOLO) detector paradigms.
5.	Semantic, instance, and panoptic segmentation techniques for pixel-level scene understanding and data masking.
6.	Motion analysis and tracking algorithms including optical flow estimation and Kalman filtering for videos.
7.	3D Computer Vision: Stereo vision, structure-from-motion, and simultaneous localization and mapping (SLAM) foundations.
8.	Perceptive AI in practice: Human pose estimation, facial recognition, and behavioral analysis in streams.
9.	Multi-modal perception: Integrating visual data with LiDAR, radar, and depth sensors for robotics.
10.	Ethical considerations in computer vision: Algorithmic fairness, privacy-preserving perception, and visual bias mitigation.

Learning activities

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Implementation-focused tasks involving Python-based vision libraries (OpenCV, PyTorch) for algorithmic validation.
Group Work	Collaborative research projects focusing on implementing a perceptive AI module for autonomous systems.
Online Activities	Accessing digital datasets (COCO, ImageNet), research papers, and JupyterHub environments for GPU training.

Activities	Explanation
Self-study	Independent literature review and coding development accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this specialization module.
Lab or Workshop	Integrated sessions using faculty GPU clusters for training large-scale vision models.
Field Work	Not applicable for this specialization module.

Mode of assessment

Assignments (20 %): four individual assignments implementing and evaluating vision pipelines on standard image and video datasets, graded individually.
Group project (30 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.
Final examination (50 %): written examination of 90 to 120 minutes, closed book; no materials or references are permitted in the examination room.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final exam	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Szeliski, R. (2022). *Computer Vision: Algorithms and Applications*, 2nd ed. Springer.	**Required**	
Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.	**Required**	
Forsyth, D. A., & Ponce, J. (2011). *Computer Vision: A Modern Approach*, 2nd ed. Pearson.	Recommended	

Material	Type	Material Locations
Prince, S. J. D. (2012). *Computer Vision: Models, Learning, and Inference*. Cambridge University	Recommended	
Klette, R. (2014). *Concise Computer Vision*. Springer.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

MODULE DESCRIPTION

Module title	**Specialisation 4: Robotics & Physical AI**
Module code	
Study program	**Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Nguyen Tuan Cuong	cuong.nt2@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☒ Compulsory Elective ☐ Elective

Semester 2

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Core 3 Deep Learning and Specialisation 3 Computer Vision & Perceptive AI, together with multivariable calculus, linear algebra and proficiency in Python or C++.

Intended learning outcomes

On successful completion of this module the learner will be able to:

1. Analyze the forward and inverse kinematics of multi-degree-of-freedom robotic manipulators using Denavit-Hartenberg conventions.
2. Design complex control architectures that integrate real-time sensor feedback for stable robotic interaction in unstructured environments.
3. Evaluate the performance of SLAM and path-planning algorithms against computational constraints and physical safety requirements.
4. Critically synthesize computer vision and reinforcement learning techniques to develop "Physical AI" agents capable of dexterous manipulation.

Contents

No.	Topic
1.	Spatial descriptions and transformations: rotation matrices, Euler angles, quaternions, and homogeneous transformation matrices.
2.	Forward and Inverse Kinematics: geometric and algebraic solutions for multi-joint robotic arm configurations.
3.	Robot Dynamics: Lagrangian and Newton-Euler formulations for modeling joint torques and inertial forces.
4.	Path planning and trajectory generation: cubic splines, A* search, and Rapidly-exploring Random Trees (RRT).
5.	Sensing and Perception: LIDAR processing, depth-camera integration, and point cloud registration for environment mapping.
6.	Control systems: PID control, Computed Torque Control, and impedance control for physical human-robot interaction.
7.	Robot Operating System (ROS2): distributed software architectures, nodes, topics, and real-time messaging middleware.
8.	Simultaneous Localization and Mapping (SLAM): Extended Kalman Filters and graph-based optimization for navigation.
9.	Physical AI: End-to-end learning for robotics, Sim-to-Real transfer, and imitation learning for complex tasks.
10.	Safety and Ethics: ISO 10218 standards, collaborative robot safety, and the societal impact of automation.

Learning activities *(Learning and teaching methods have to be described (e.g. lectures, exercises, seminars, internships, project work, self-study)*

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Periodic technical reports and coding exercises focusing on kinematic modeling and control loops.
Group Work	Collaborative engineering project focusing on deploying an autonomous navigation stack in a simulation.
Online Activities	Accessing Gazebo/Webots simulations, digital documentation (ROS2), and collaborative code repositories.

Activities	Explanation
Self-study	Independent research, algorithm development, and technical documentation accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this specialization module.
Lab or Workshop	Integrated sessions using the faculty's robotic manipulators and mobile platforms for hardware validation.
Field Work	Not applicable for this specialization module.

Mode of assessment

Assignments (20 %): four individual assignments implementing perception, planning and control components in simulation and on the faculty robotics platforms, graded individually.
Group project (30 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.
Practical examination (50 %): an individual laboratory examination of 60 to 90 minutes on the faculty robotics platform, in which the candidate implements, integrates and demonstrates a specified perception-and-control task and answers questions on the design decisions taken. A closed-book written examination is not used, because the intended learning outcomes of this module are demonstrated on a physical system.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Practical examination	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Spong, M. W., Hutchinson, S., & Vidyasagar, M. (2020). *Robot Modeling and Control*, 2nd ed. Wiley.	**Required**	
Lynch, K. M., & Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*, 1st ed. Cambridge University Press.	**Required**	

Material	Type	Material Locations
Siciliano, B., & Khatib, O. (2016). *Springer Handbook of Robotics*, 2nd ed. Springer.	Recommended	
Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to Autonomous Mobile Robots*, 2nd ed. MIT Press.	Recommended	
Quigley, M., Gerkey, B., & Smart, W. D. (2015). *Programming Robots with ROS*, 1st ed. O'Reilly Media.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

MODULE DESCRIPTION

Module title	**Elective 1: Reinforcement Learning**
Module code	
Study program	**Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Truong Dinh Huy	huy.td@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☐ Compulsory Elective ☒ Elective

Semester 3

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Core 3 Deep Learning, together with probability, linear algebra, multivariable calculus and proficiency in Python.

Intended learning outcomes

On successful completion of this module the learner will be able to:

1. Formalize sequential decision-making problems as Markov decision processes and partially observable Markov decision processes, justifying the chosen state, action, reward, and discount specification.
2. Derive and implement value-based, policy-gradient, and actor-critic algorithms, and analyze their convergence behaviour, bias-variance trade-offs, and sample complexity.
3. Design deep reinforcement learning systems for high-dimensional environments, selecting appropriate function approximators, exploration strategies, and stabilization techniques.

4. Evaluate reinforcement learning agents empirically under reproducibility, safety, and reward-specification constraints, including offline and preference-based settings such as reinforcement learning from human feedback.
5. Critically assess the applicability, operational risks, and ethical implications of deploying autonomous learning agents in industrial and societal environments.

Contents *(Outline of subject and topics explored)*

No.	Topic
1.	Sequential decision making: Markov decision processes, returns, policies, value functions, and the Bellman equations.
2.	Dynamic programming: policy evaluation, policy iteration, value iteration, and generalized policy iteration.
3.	Model-free prediction and control: Monte Carlo methods, temporal-difference learning, SARSA, Q-learning, and eligibility traces.
4.	Function approximation: linear methods, the deadly triad, experience replay, and target networks.
5.	Deep value-based methods: DQN and its extensions (double, duelling, prioritized replay, distributional RL, Rainbow).
6.	Policy-gradient and actor-critic methods: REINFORCE, A2C/A3C, TRPO, PPO, DDPG, TD3, and soft actor-critic.
7.	Exploration: epsilon-greedy strategies, upper-confidence bounds, Thompson sampling, count-based and intrinsic-motivation methods.
8.	Model-based reinforcement learning and planning: world models, Dyna, Monte Carlo tree search, and AlphaZero-style search.
9.	Offline, multi-agent, and preference-based reinforcement learning, including reinforcement learning from human feedback for foundation models.
10.	Reproducibility, safety, and applications: benchmark suites, evaluation protocols, reward hacking, safe exploration, and robotics and operations use cases.

Learning activities *(Learning and teaching methods have to be described (e.g. lectures, exercises, seminars, internships, project work, self-study)*

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Implementation of tabular and deep reinforcement learning agents in Gymnasium-style environments, with reported learning curves and ablations.
Group Work	A team project training and evaluating a deep reinforcement learning agent on a control or game benchmark against a documented baseline.
Online Activities	Accessing simulation environments, GPU training resources, and recorded tutorial sessions.
Self-study	Independent reading and assignments accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this elective module.
Lab or Workshop	Integrated laboratory sessions on Gymnasium, Stable-Baselines3, and GPU-accelerated agent training.
Field Work	Not applicable for this elective module.

Mode of assessment

Assignments (20 %): four individual assignments implementing and evaluating value-based and policy-gradient agents in standard simulation environments, graded individually.
Group project (30 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.
Final examination (50 %): written examination of 90 to 120 minutes, closed book; no materials or references are permitted in the examination room.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final exam	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction, 2nd ed. MIT Press.	**Required**	
Szepesvari, C. (2010). Algorithms for Reinforcement Learning. Morgan & Claypool.	**Required**	
Lattimore, T., & Szepesvari, C. (2020). Bandit Algorithms. Cambridge University Press.	Recommended	
Bertsekas, D. P. (2019). Reinforcement Learning and Optimal Control. Athena Scientific.	Recommended	
Levine, S., Kumar, A., Tucker, G., & Fu, J. (2020). Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems. arXiv:2005.01643.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

MODULE DESCRIPTION

Module title Elective 2: Multi-Agent Systems & Agentic AI
Module code
Study program Master of Computer Science

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Tran Duc Khanh	khanh.td@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☐ Compulsory Elective ☒ Elective

Semester 3

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Core 2 Advanced Algorithms and Core 3 Deep Learning, together with probability theory and proficiency in Python.

Intended learning outcomes

On successful completion of this module the learner will be able to:
1. Analyze the formal properties of multi-agent interactions using game-theoretic frameworks, including Nash equilibrium and mechanism design.
2. Design and implement agentic architectures utilizing Large Language Models (LLMs) for autonomous planning, tool-use, and multi-step reasoning.
3. Evaluate the coordination and communication protocols within decentralized systems to ensure convergence and stability in competitive and cooperative environments.
4. Critically synthesize distributed problem-solving strategies to architect scalable multi-agent systems for real-world robotics or digital twin simulations.

Contents

No.	Topic
1.	Foundations of intelligent agents: architectures, environments, and the spectrum of autonomous agentic behavior.
2.	Multi-agent interaction: strategic games, equilibrium concepts, and the mechanics of competitive decision-making.
3.	Coordination and collaboration: communication protocols, consensus algorithms, and decentralized task allocation strategies.
4.	Auctions and mechanism design: designing incentive-compatible systems for resource allocation and social welfare.
5.	Multi-agent Reinforcement Learning (MARL): joint action spaces, credit assignment, and non-stationarity in learning environments.
6.	Agentic AI with LLMs: autonomous reasoning loops, memory management, and advanced tool-augmentation for agents.
7.	Multi-agent planning: partial observability, hierarchical planning, and collective intent in dynamic and uncertain worlds.
8.	Negotiation and argumentation: formal models of bargaining, conflict resolution, and automated negotiation between agents.
9.	Agentic workflows and swarms: designing emergent behavior through simple local rules and global synchronization.
10.	Security and Ethics in Agentic AI: addressing multi-agent safety, alignment, and accountability in autonomous systems.

Learning activities *(Learning and teaching methods have to be described (e.g. lectures, exercises, seminars, internships, project work, self-study)*

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Periodic technical reports and coding exercises focusing on specific agent architectures and game-theoretic models.
Group Work	Collaborative project involving the development of a decentralized multi-agent system or agentic AI workflow.
Online Activities	Accessing simulation environments (Gymnasium, PettingZoo), research repositories, and collaborative version control platforms.
Self-study	Independent research, academic paper reading, and software development accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this elective module.
Lab or Workshop	Integrated sessions focusing on building and testing LLM-based agents in simulated multi-agent environments.
Field Work	Not applicable for this elective module.

Mode of assessment

Assignments (20 %): four individual assignments implementing agent architectures, coordination protocols and tool-using agents, graded individually.
Group project (30 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.
Final examination (50 %): written examination of 90 to 120 minutes, closed book; no materials or references are permitted in the examination room.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final exam	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Shoham, Y., & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press.	**Required**	
Wooldridge, M. (2009). *An Introduction to MultiAgent Systems*, 2nd ed. John Wiley & Sons.	**Required**	
Albrecht, S. V., Christianos, F., & Schäfer, L. (2024). *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press.	Recommended	
Nisan, N., Roughgarden, T., Tardos, É., & Vazirani, V. V. (Eds.) (2007). Algorithmic Game Theory. Cambridge University Press.	Recommended	
Selected Research Papers (Agentic Workflows, LLM-based Autonomous Agents, and Swarm Intelligence).	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

MODULE DESCRIPTION

Module title Elective 3: Distributed Systems
Module code
Study program Master of Computer Science

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Tran Huu Tam	tam.th@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☐ Compulsory Elective ☒ Elective

Semester 3

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Core 4 High Performance Computing, operating systems and computer networks, and proficiency in at least one high-level programming language.

Intended learning outcomes

On successful completion of this module the learner will be able to:
1. Analyse the correctness of a distributed computation using appropriate models of timing, failure and consistency.
2. Explain and compare consensus and replication protocols, and justify the protocol chosen for a stated fault model.
3. Design a partitioned and replicated data service, and state the consistency and availability trade-offs accepted.
4. Implement and test a distributed protocol, and demonstrate its behaviour under injected faults.

5. Evaluate the dependability and the scalability of a distributed system using observability, tracing and controlled fault injection.

Contents

No.	Topic
1.	Models of distributed computation: synchronous, asynchronous and partially synchronous systems; crash, omission and Byzantine failure models; and the FLP impossibility result with its practical consequences.
2.	Time, ordering and coordination: physical and logical clocks, vector clocks, causal ordering, consistent global snapshots and distributed mutual exclusion.
3.	Consensus and state-machine replication: Paxos, Raft, leader election, view changes and quorum systems.
4.	Consistency models: linearizability, sequential and causal consistency, eventual consistency, the CAP and PACELC formulations, and conflict-free replicated data types.
5.	Distributed data systems: partitioning and sharding, replication strategies, distributed key-value and NewSQL stores, and distributed file systems.
6.	Transactions across nodes: atomic commit, two-phase and three-phase commit, snapshot isolation, and the cost of distributed coordination.
7.	Communication and interaction models: remote procedure call, publish-subscribe, message queues, streaming platforms and back-pressure.
8.	Dependability engineering: failure detectors, timeouts and retries, idempotence, circuit breakers, fault injection and chaos experiments.
9.	Observability and operations: distributed tracing, metrics and structured logging, capacity planning, and the measurement of tail latency.
10.	Security and trust in distributed settings: authentication between services, secure channels, and an introduction to Byzantine fault tolerance and permissioned ledgers.

Learning activities *(Learning and teaching methods have to be described (e.g. lectures, exercises, seminars, internships, project work, self-study)*

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Programming assignments implementing and testing distributed protocols, including replication, leader election and consensus, each submitted with a short evaluation report.
Group Work	Collaborative project building a partitioned and replicated data service, evaluated under injected faults and reported with a tracing study.
Online Activities	Accessing container and orchestration environments, tracing and observability tools, research repositories and collaborative version control repositories.
Self-study	Independent research, academic paper reading and software development accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this elective module.
Lab or Workshop	Integrated laboratory sessions on a multi-node cluster: protocol implementation, fault injection and distributed tracing.
Field Work	Not applicable for this elective module.

Mode of assessment

Assignments (20 %): four individual assignments implementing and testing distributed protocols - logical clocks, replication, consensus and a fault-injection study - each submitted with a short evaluation report and graded individually.
Group project (30 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.
Final examination (50 %): written examination of 90 to 120 minutes, closed book; no materials or references are permitted in the examination room.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final exam	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Tanenbaum, A. S., & Van Steen, M. (2023). Distributed Systems, 4th ed.	**Required**	
Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.	**Required**	
Cachin, C., Guerraoui, R., & Rodrigues, L. (2011). Introduction to Reliable and Secure Distributed Programming, 2nd ed. Springer.	Recommended	
Ongaro, D., & Ousterhout, J. (2014). In Search of an Understandable Consensus Algorithm. USENIX Annual Technical Conference.	Recommended	
Burns, B. (2018). Designing Distributed Systems. O'Reilly Media.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

MODULE DESCRIPTION

Module title Elective 4: MLOps and AI Deployment
Module code
Study program Master of Computer Science

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Nguyen Tuan Cuong	cuong.nt2@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☐ Compulsory Elective ☒ Elective

Semester 3

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Core 4 High Performance Computing and Core 5 Data Engineering, together with proficiency in Python, containerisation and version control.

Intended learning outcomes

On successful completion of this module the learner will be able to:
1. Design an end-to-end machine-learning system, from data ingestion and feature management through training, evaluation, deployment and monitoring.
2. Implement reproducible training and deployment pipelines using containers, workflow orchestration and continuous integration and delivery.
3. Select and justify a serving architecture for a stated latency, throughput and cost budget, and state the trade-offs accepted.
4. Detect and diagnose data drift, concept drift and model degradation in production, and specify the retraining policy that follows.

5. Evaluate a deployed system against reliability, cost and governance requirements, and document the result reproducibly.

Contents

No.	Topic
1.	The machine-learning system life cycle: problem framing, success metrics, and the distinction between model performance and system performance.
2.	Data and feature management: data validation, feature stores, training-serving skew, and the reproducibility of datasets.
3.	Experiment management: experiment tracking, versioning of data, code and models, and the model registry.
4.	Training pipelines: workflow orchestration, scheduling, distributed training jobs, and resource management on shared infrastructure.
5.	Packaging and containerisation: images, dependency management, and reproducible runtime environments.
6.	Deployment and serving: batch, online and streaming inference; microservice and edge deployment; autoscaling; latency and cost budgets.
7.	Release strategies: shadow deployment, canary release, A/B testing, and safe rollback.
8.	Monitoring and observability: service metrics, model metrics, data and concept drift detection, alerting, and incident response.
9.	Continuous integration, delivery and training: automated testing of data and models, and the design of retraining triggers.

10.	Governance and technical debt: lineage, auditability, model documentation, and the hidden technical debt of machine-learning systems.

Learning activities
(Learning and teaching methods have to be described (e.g. lectures, exercises, seminars, internships, project work, self-study)

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Programming assignments building a training pipeline, a containerised serving endpoint, and a drift-monitoring component, each submitted with a short evaluation report.
Group Work	Collaborative project deploying and operating a complete machine-learning service, including a release strategy and a monitoring dashboard.

Draft

Activities	Explanation
Online Activities	Accessing container and orchestration environments, experiment-tracking tools, and collaborative version control repositories.
Self-study	Independent research, academic paper reading and software development accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this elective module.
Lab or Workshop	Integrated laboratory sessions on the faculty cluster: pipeline construction, deployment, load testing and drift injection.
Field Work	Not applicable for this elective module.

Mode of assessment
Assignments (20 %): four individual assignments covering a reproducible training pipeline, a containerised serving endpoint, a monitoring and drift-detection component, and a cost and latency study, each submitted with a short report and graded individually.
Group project (30 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.
Final examination (50 %): written examination of 90 to 120 minutes, closed book; no materials or references are permitted in the examination room.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final exam	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Huyen, C. (2022). Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. O'Reilly Media.	**Required**	
Lakshmanan, V., Robinson, S., & Munn, M. (2020). Machine Learning Design Patterns. O'Reilly Media.	**Required**	

Draft

Material	Type	Material Locations
Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.	Recommended	
Burns, B. (2018). Designing Distributed Systems. O'Reilly Media.	Recommended	
Sculley, D., Holt, G., Golovin, D., et al. (2015). Hidden Technical Debt in Machine Learning Systems. Advances in Neural Information Processing Systems 28.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

Draft

MODULE DESCRIPTION

Module title **Elective 5: Security & Privacy**
Module code
Study program **Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Tran Hong Ngoc	ngoc.th@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☐ Compulsory Elective ☒ Elective

Semester 3

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Core 5 Data Engineering, operating systems and computer networks, and proficiency in at least one systems programming language.

Intended learning outcomes
On successful completion of this module the learner will be able to:

1. Analyze complex software and network architectures to identify latent vulnerabilities, attack vectors, and systemic security risks.
2. Design robust defense-in-depth strategies incorporating advanced cryptographic protocols, intrusion detection systems, and secure coding practices.
3. Evaluate the effectiveness of modern security frameworks and mitigation techniques against sophisticated, multi-stage cyber-physical threats.
4. Critically synthesize ethical hacking methodologies and risk management principles to ensure the resilience of AI-integrated critical infrastructures.

Draft

Contents

No.	Topic
1.	Formal foundations of computer security: confidentiality, integrity, availability, and advanced threat modeling.
2.	Applied Cryptography: symmetric and asymmetric primitives, digital signatures, and public key infrastructures.
3.	Network security protocols: securing the transport layer, DNSSEC, VPNs, and firewall configuration strategies.
4.	Software security and exploitation: buffer overflows, format string vulnerabilities, and modern memory protection.
5.	Web application security: cross-site scripting (XSS), SQL injection, and session management hijacking techniques.
6.	Operating system security: access control lists, kernel hardening, sandboxing, and virtualization security mechanisms.
7.	Intrusion detection and prevention systems: anomaly-based detection and signature matching in high-speed networks.
8.	Security for AI and Machine Learning: adversarial attacks, model poisoning, and privacy-preserving computation.
9.	Digital forensics and incident response: log analysis, memory forensics, and post-mortem artifact recovery.
10.	Cyber-physical systems and IoT security: protecting hardware-constrained devices and critical industrial control systems.

Learning activities
(Learning and teaching methods have to be described (e.g. lectures, exercises, seminars, internships, project work, self-study)

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Applied Cryptography: symmetric and asymmetric primitives, digital signatures, and public key infrastructures.
Group Work	Network security protocols: securing the transport layer, DNSSEC, VPNs, and firewall configuration strategies.
Online Activities	Software security and exploitation: buffer overflows, format string vulnerabilities, and modern memory protection.

Draft

Activities	Explanation
Self-study	Web application security: cross-site scripting (XSS), SQL injection, and session management hijacking techniques.
Internship	Operating system security: access control lists, kernel hardening, sandboxing, and virtualization security mechanisms.
Lab or Workshop	Intrusion detection and prevention systems: anomaly-based detection and signature matching in high-speed networks.
Field Work	Security for AI and Machine Learning: adversarial attacks, model poisoning, and privacy-preserving computation.

Mode of assessment
Assignments (20 %): four individual assignments analysing and hardening provided systems and datasets, documented in short technical reports and graded individually.
Group project (30 %): a project carried out in teams of three to four students producing a software artefact together with a written report of 10 to 15 pages, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.
Final examination (50 %): written examination of 90 to 120 minutes, closed book; no materials or references are permitted in the examination room.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final exam	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Stallings, W., & Brown, L. (2024). Computer Security: Principles and Practice, 5th ed. Pearson.	**Required**	
Anderson, R. (2020). *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed. Wiley.	**Required**	
Kaufman, C., Perlman, R., Speciner, M., &	Recommended	

Draft

Material	Type	Material Locations
Perlner, R. (2022). *Network Security: Private Communication in a Public World*, 3rd ed. Addison-Wesley Professional (Pearson).		
Erickson, J. (2008). *Hacking: The Art of Exploitation*, 2nd ed. No Starch Press.	Recommended	
Bishop, M. (2018). Computer Security: Art and Science, 2nd ed. Addison-Wesley.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

Draft

MODULE DESCRIPTION

Module title **Elective 6: AI Law, Ethics and Policy**
Module code
Study program **Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Dinh Hai Dung	dung.dh@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☐ Compulsory Elective ☒ Elective

Semester 3

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Students are expected to have a solid foundation in Artificial Intelligence, Data Engineering, and proficiency in analyzing technical documentation or research papers. Knowledge of current global technology trends and basic philosophical or legal frameworks is highly recommended to engage with complex discussions on algorithmic bias and international AI regulation.

Intended learning outcomes
On successful completion of this module the learner will be able to:

1. Analyze the multi-faceted ethical implications of automated decision-making systems, including fairness, transparency, and accountability in high-stakes environments.
2. Design technical and procedural mitigation strategies to address algorithmic bias and ensure privacy-preserving data processing in AI pipelines.

Draft

---

3. Evaluate current and emerging international AI regulatory frameworks (e.g., EU AI Act) against corporate compliance and technical feasibility.
4. Critically synthesize the intersection of intellectual property law, liability models, and the societal impact of generative AI on labor and democracy.

Contents
(Outline of subject and topics explored)

No.	Topic
1.	Foundations of normative ethics: utilitarianism, deontology, and virtue ethics applied to autonomous systems.
2.	Algorithmic fairness: mathematical definitions of bias, parity metrics, and techniques for disparate impact mitigation.
3.	Transparency and Explainability (XAI): technical requirements for model interpretability in regulated and judicial sectors.
4.	Privacy and Data Protection: Deep dive into GDPR, data sovereignty, and differential privacy mechanisms.
5.	Global AI Governance: Comparative analysis of the EU AI Act, US Executive Orders, and UNESCO recommendations.
6.	Intellectual Property and AI: Copyright challenges in training data, fair use, and ownership of synthetic outputs.
7.	Liability and Accountability: Legal personhood, tort law for autonomous agents, and the "black box" liability problem.
8.	Social Impact of AI: Automation's effect on labor markets, digital divide, and the erosion of cognitive autonomy.
9.	AI Safety and Existential Risk: Technical alignment, value loading, and the governance of Artificial General Intelligence (AGI).
10.	Policy-by-Design: Integrating ethical and legal constraints directly into the software development and MLOps lifecycle.

Learning activities
(Learning and teaching methods have to be described (e.g. lectures, exercises, seminars, internships, project work, self-study)

Activities	Explanation
Attendance	Attendance at all sessions is expected. Attendance and participation in class discussion are not graded and form no part of any assessment component.
Individual Assignments	Periodic technical reports and case study analyses focusing on specific AI ethical dilemmas or policies.
Group Work	Collaborative "Policy Mock-ups" where students draft compliance

Draft

Activities	Explanation
	documents for hypothetical AI startups.
Online Activities	Accessing digital regulatory archives, research repositories, and collaborative debate platforms.
Self-study	Independent research, reading of legal texts, and technical policy development accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this elective module.
Lab or Workshop	Integrated sessions focusing on utilizing bias-detection toolkits and privacy-auditing software on real datasets.
Field Work	Not applicable for this elective module.

Mode of assessment
Assignments (20 %): four individual written analyses of 1,500 to 2,000 words each, applying an international regulatory instrument to a given technical case, graded individually.
Group project (30 %): a written conformity or impact assessment of 15 to 20 pages prepared in teams of three to four students on an assigned AI system, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the report, a documented statement of individual contribution and the individual questioning.
Take-home case analysis (50 %): an individual written analysis of an unseen case of 2,500 to 3,000 words, prepared within 48 hours, open book, requiring the application of the relevant instruments to the facts and a reasoned conclusion. A closed-book written examination is not used, because the intended learning outcomes of this module concern the application of regulatory instruments rather than their recall.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Take-home case analysis	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Coeckelbergh, M. (2020). *AI Ethics*, 1st ed. MIT Press.	**Required**	

Draft

Material	Type	Material Locations
Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). OJ L, 12.7.2024.	**Required**	
Dubber, M. D., Pasquale, F., & Das, S. (2020). *The Oxford Handbook of Ethics of AI*. Oxford University Press.	Recommended	
Barocas, S., Hardt, M., & Narayanan, A. (2023). *Fairness and Machine Learning: Limitations and Opportunities*. MIT Press.	Recommended	
O'Neil, C. (2016). *Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy*. Crown.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

Draft

MODULE DESCRIPTION

Module title **Elective 7: Mathematical Aspects of Differential Equations and Numerical Mathematics**
Module code
Study program **Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Nguyen Thien Binh	binh.nt@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☐ Compulsory Elective ☒ Elective

Semester 3

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites Core 1 Mathematics of Computing, together with undergraduate analysis and linear algebra and proficiency in Python or MATLAB.

Intended learning outcomes
On successful completion of this module the learner will be able to:

1. Classify ordinary and partial differential equations and state the conditions under which a solution exists and is unique.
2. Derive numerical methods for differential equations and analyse their consistency, stability and convergence.
3. Select an appropriate discretisation and solver for a stated problem and justify the choice against accuracy and cost requirements.
4. Quantify and control numerical error, including truncation error, round-off and the effect of conditioning.

Draft

---

5. Implement and validate a numerical solver and report its observed convergence against an analytical or reference solution.

Contents

No.	Topic
1.	Ordinary differential equations: initial-value problems, existence and uniqueness, well-posedness, and stiffness.
2.	One-step methods: explicit and implicit Euler, Runge-Kutta families, order conditions, and local and global error.
3.	Multistep methods and stability: consistency, zero-stability, regions of absolute stability, and solvers for stiff systems.
4.	Boundary-value problems: shooting methods, finite differences, and collocation.
5.	Partial differential equations: classification into elliptic, parabolic and hyperbolic problems and the behaviour each implies.
6.	Finite-difference discretisation: consistency, the Courant-Friedrichs-Lewy condition, von Neumann stability analysis, and the Lax equivalence theorem.
7.	Finite-element and spectral methods: weak formulations, basis functions, and a priori error estimates.
8.	Numerical linear algebra for discretised systems: sparse direct methods, conditioning, Krylov subspace iterations, and preconditioning.
9.	Floating-point arithmetic and error analysis: round-off, cancellation, backward stability, and conditioning.
10.	Verification and validation: the method of manufactured solutions, convergence studies, and reproducible numerical experiments.

Learning activities
(Learning and teaching methods have to be described (e.g. lectures, exercises, seminars, internships, project work, self-study)

Activities	Explanation
Attendance	Attendance at the scheduled sessions, supervision meetings and colloquia is expected. Attendance and participation are not graded and form no part of any assessment component.
Individual Assignments	Problem sets combining analytical derivations of stability and convergence with numerical implementations and convergence studies.
Group Work	Collaborative project solving an applied differential-equation problem numerically, including verification against a reference solution.
Online Activities	Accessing numerical computing environments, Jupyter notebooks and

Draft

Activities	Explanation
	recorded tutorials.

Self-study	Independent study, exercise work and software development accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this elective module.
Lab or Workshop	Integrated computer laboratory sessions on solver implementation, stability experiments and convergence measurement.
Field Work	Not applicable for this elective module.

Mode of assessment

Assignments (20 %): four individual problem sets combining analytical derivations of consistency, stability and convergence with numerical implementations and documented convergence studies, graded individually.
Group project (30 %): a project carried out in teams of three to four students solving an applied differential-equation problem numerically, delivering verified solver code together with a written report of 10 to 15 pages including a verification study against an analytical or reference solution, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.
Final examination (50 %): written examination of 90 to 120 minutes, closed book, comprising analytical questions on discretisation, stability and error analysis together with the interpretation of given numerical output; no materials or references are permitted in the examination room.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final examination	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Iserles, A. (2008). A First Course in the Numerical Analysis of Differential Equations, 2nd ed. Cambridge University Press.	**Required**	

Draft

Material	Type	Material Locations
LeVeque, R. J. (2007). Finite Difference Methods for Ordinary and Partial Differential Equations. SIAM.	**Required**	
Trefethen, L. N., & Bau, D. (1997). Numerical Linear Algebra. SIAM.	Recommended	
Quarteroni, A., Sacco, R., & Saleri, F. (2007). Numerical Mathematics, 2nd ed. Springer.	Recommended	
Sueli, E., & Mayers, D. F. (2003). An Introduction to Numerical Analysis. Cambridge University Press.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

Draft

MODULE DESCRIPTION

Module title	**Elective 8: Machine Learning Techniques for Engineering Applications**
Module code	
Study program	**Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Nguyen Thien Binh	binh.nt@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☐ Compulsory ☐ Compulsory Elective ☒ Elective

Semester 3

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	6	ECTS
Contact hours	60	AHs
Assignments and independent learning	120	AHs
Total Working hours	180	AHs

Frequency The module is offered once each academic year.

Prerequisites **Core 3 Deep Learning, with Core 1 Mathematics of Computing recommended, together with proficiency in Python.**

Intended learning outcomes

On successful completion of this module the learner will be able to:

1. Select machine-learning techniques appropriate to engineering data, accounting for small samples, measurement noise, physical constraints and the cost of data acquisition.
2. Build and validate surrogate models of expensive simulations and quantify their predictive uncertainty.
3. Incorporate physical knowledge into a learning model and evaluate the effect on data efficiency and generalisation.
4. Apply machine learning to engineering tasks such as condition monitoring, fault diagnosis, design optimisation and control.

Draft

---

5. Evaluate an engineering machine-learning model against domain requirements, including reliability, interpretability and behaviour outside the training range.

Contents

No.	Topic
1.	Engineering data: sensors and time series, design of experiments, small-sample regimes, measurement noise and missing data.
2.	Regression and classification for engineering data: regularised linear models, kernel methods and tree ensembles, and the regimes in which they outperform deep models.
3.	Gaussian processes and uncertainty quantification: predictive distributions, hyperparameter learning and calibration.
4.	Surrogate modelling of expensive simulations: sampling strategies and the accuracy-cost trade-off.
5.	Bayesian optimisation for design and parameter tuning under expensive evaluations.
6.	Dimensionality reduction and reduced-order modelling: proper orthogonal decomposition, dynamic mode decomposition, and manifold methods.
7.	Physics-informed machine learning: physics-informed neural networks, embedding of constraints and conservation laws, and hybrid physical-statistical models.
8.	Time-series and signal models for condition monitoring, anomaly detection and predictive maintenance.
9.	System identification and data-driven control: learning dynamics from data and using the learned model in a control loop.
10.	Deployment in engineering settings: interpretability, extrapolation risk, validation against physical tests, and certification constraints.

Learning activities

Activities	Explanation
Attendance	Attendance at the scheduled sessions, supervision meetings and colloquia is expected. Attendance and participation are not graded and form no part of any assessment component.
Individual Assignments	Assignments on surrogate modelling, uncertainty quantification and physics-informed learning, each applied to an engineering dataset and submitted with a short evaluation report.

Draft

Activities	Explanation
Group Work	Collaborative project applying machine learning to an engineering problem, including a validation study against physical or simulated reference data.
Online Activities	Accessing datasets, simulation environments, Jupyter notebooks and collaborative version control repositories.
Self-study	Independent research, academic paper reading and software development accounting for 120 Academic Hours (AHs).
Internship	Not applicable for this elective module.
Lab or Workshop	Integrated laboratory sessions on surrogate construction, uncertainty calibration and physics-informed model training.
Field Work	Not applicable for this elective module.

Mode of assessment

Assignments (20 %): four individual assignments on surrogate modelling, uncertainty quantification and physics-informed learning, each applied to an engineering dataset and submitted with a short evaluation report, graded individually.
Group project (30 %): a project carried out in teams of three to four students applying machine learning to an engineering problem, delivering reproducible code together with a written report of 10 to 15 pages including a validation study against physical or simulated reference data, concluded by a team presentation of 15 minutes with individual questioning; the individual mark is derived from the artefact, the report, a documented statement of individual contribution and the individual questioning.
Final examination (50 %): written examination of 90 to 120 minutes, closed book, covering the selection of methods for engineering data, surrogate and physics-informed modelling, uncertainty quantification and the evaluation of models against domain requirements; no materials or references are permitted in the examination room.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Assignments	20%
Group project	30%
Final examination	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Draft

Material	Type	Material Locations
Brunton, S. L., & Kutz, J. N. (2022). Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control, 2nd ed. Cambridge University Press.	**Required**	
Rasmussen, C. E., & Williams, C. K. I. (2006). Gaussian Processes for Machine Learning. MIT Press.	**Required**	
Geron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. O'Reilly Media.	Recommended	
Forrester, A., Sobester, A., & Keane, A. (2008). Engineering Design via Surrogate Modelling: A Practical Guide. Wiley.	Recommended	
Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks. Journal of Computational Physics, 378, 686-707.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

Draft

MODULE DESCRIPTION

Module title	**Advanced Seminar**
Module code	
Study program	**Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Tran Duc Khanh	khanh.td@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☒ Compulsory ☐ Compulsory Elective ☐ Elective

Semester 2

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	3	ECTS
Contact hours	30	AHs
Assignments and independent learning	60	AHs
Total Working hours	90	AHs

Frequency The module is offered once each academic year.

Prerequisites None. The module introduces the reading, critical appraisal and oral presentation of the research literature and assumes no prior training in research methods. The systematic preparation of scientific manuscripts is treated in Interdisciplinary Key Qualification 1 Scientific Writing and Research; the two modules are delimited so that the Advanced Seminar addresses reading and critique of the literature, and Interdisciplinary Key Qualification 1 addresses writing, documentation and research-project design.

Intended learning outcomes

On successful completion of this module the learner will be able to:

1. Analyze and critically critique state-of-the-art research papers to identify significant gaps, methodological limitations, and novel contributions in AI.
2. Design a comprehensive research proposal incorporating a rigorous hypothesis, defined problem statement, and appropriate validation frameworks for research.
3. Evaluate the ethical dimensions and societal impacts of emerging AI technologies, focusing on algorithmic transparency and data privacy standards.

Draft

---

4. Critically synthesize complex technical information into high-quality scientific manuscripts and professional oral presentations for specialized academic audiences.

Contents

No.	Topic
1.	Finding and scoping a research question in Computer Science and Artificial Intelligence.
2.	Literature search and systematic mapping: databases, citation analysis, and identifying state-of-the-art benchmarks.
3.	Reading a research paper: claims, method, evidence, threats to validity, and reproducibility.
4.	Critical appraisal: conducting peer reviews and giving constructive feedback on technical manuscripts.
5.	Comparing methods: baselines, ablation studies, and the interpretation of reported results.
6.	Identifying research gaps and open problems from a body of literature.
7.	Structuring a seminar contribution: from a set of papers to a coherent argument.
8.	Visual communication of research: figures, tables and charts for a specialist audience.
9.	Presentation skills: architecting a technical talk and handling questions in an academic seminar.
10.	Research ethics in reading and citing: plagiarism, attribution, and the disclosure of generative AI tools.

Learning activities

(Learning and teaching methods have to be described (e.g. lectures, exercises, seminars, internships, project work, self-study)

Activities	Explanation
Attendance	Attendance at the scheduled sessions, supervision meetings and colloquia is expected. Attendance and participation are not graded and form no part of any assessment component.
Individual Assignments	Weekly reading summaries, peer-review reports, and a final scientific review paper.
Group Work	Collaborative "Lab Meetings" focusing on brainstorming research methodologies and pilot study designs.
Online Activities	Accessing digital libraries (ACM/IEEE), research repositories, and collaborative writing platforms (Overleaf).

Draft

Activities	Explanation
Self-study	Independent literature research and manuscript drafting accounting for 60 Academic Hours (AHs).
Internship	Not applicable for this core research module.
Lab or Workshop	Integrated sessions on LaTeX, BibTeX management, and technical visualization tools.
Field Work	Not applicable for this core research module.

Mode of assessment

Individual project (50 %): a written critical review of 8 to 10 pages on an assigned set of research papers, prepared with a reference manager and following the citation standards of the discipline, together with one written peer-review report of 1 to 2 pages on a fellow student's draft.
Final presentation (50 %): an oral presentation of 20 to 25 minutes followed by 5 to 10 minutes of discussion, assessed on command of the literature, structure of the argument, quality of the visual material and the handling of questions.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Individual project	50%
Final presentation	50%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
J. Zobel, *Writing for Computer Science*, 3rd ed. Springer, 2014	**Required**	
W. Booth et al., *The Craft of Research*, 4th ed. University of Chicago Press, 2016	**Required**	
Glasman-Deal, H. (2020). Science Research Writing, 2nd ed. World Scientific.	Recommended	
Wohlin, C., et al. (2012). *Experimentation in Software Engineering*, 1st ed. Springer.	Recommended	
Higham, N. J. (2020). *Handbook of Writing for the Mathematical Sciences*, 3rd ed. SIAM.	Recommended	

Draft

Written/updated by
Prof. Dr.
Date: / /2026

Draft

MODULE DESCRIPTION

Module title	**Interdisciplinary Key Qualification: Scientific Writing and Research**
Module code	
Study program	**Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Assoc. Prof. Tran Le Luu	luu.tl@vgu.edu.vn		By appointment
Lecturer	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☒ Compulsory ☐ Compulsory Elective ☐ Elective

Semester 2

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	3	ECTS
Contact hours	30	AHs
Assignments and independent learning	60	AHs
Total Working hours	90	AHs

Frequency The module is offered once each academic year.

Prerequisites **None. The module is delimited against the Advanced Seminar: the Advanced Seminar addresses the reading and critique of the research literature, and this module addresses the writing, documentation and design of research work in preparation for the Master Thesis.**

Intended learning outcomes

On successful completion of this module the learner will be able to:

1. Plan a research undertaking by formulating research questions, delimiting scope, and selecting methods appropriate to problems in computer science and artificial intelligence.
2. Conduct a systematic literature review using reproducible search, screening, and citation-management procedures.
3. Compose scientific text - abstract, related work, method, results, and discussion - to international publication standards, using LaTeX and structured revision techniques.

Draft

---

4. Apply research integrity standards, including authorship conventions, plagiarism avoidance, research data management, reproducibility, and the transparent declaration of generative AI assistance.

Contents

No.	Topic
1.	The research process in computer science: from an initial question to a hypothesis, scope, and contribution statement.
2.	Systematic literature search: databases (ACM DL, IEEE Xplore, Scopus), search strings, screening protocols, and snowballing.
3.	Reference management and citation styles; bibliographic workflows with BibTeX and reference managers.
4.	Structure and rhetoric of a scientific paper: title, abstract, introduction, related work, method, evaluation, discussion, and limitations.
5.	Clear scientific English: paragraph design, sentence economy, hedging, and common pitfalls for non-native writers.
6.	Document preparation with LaTeX: templates, cross-referencing, mathematical typesetting, and collaborative writing workflows.
7.	Presentation of quantitative results: tables, figures, statistical reporting, and the ethics of visualization.
8.	Peer review: review criteria, writing constructive reviews, and responding to reviewers.
9.	Research integrity: plagiarism, authorship and contributorship, conflicts of interest, and the declaration of AI-assisted writing.
10.	Research data management, open science, reproducibility packages, and preregistration.

Learning activities

(Learning and teaching methods have to be described (e.g. lectures, exercises, seminars, internships, project work, self-study)

Activities	Explanation
Attendance	Attendance at the scheduled sessions, supervision meetings and colloquia is expected. Attendance and participation are not graded and form no part of any assessment component.
Individual Assignments	A staged writing portfolio: research question, literature review, method section, and a full short paper.
Group Work	Structured peer-review circles in which students review and revise one another's manuscripts.

Draft

Activities	Explanation
Online Activities	Accessing digital libraries, collaborative writing platforms, and reference management tools.
Self-study	Independent literature research and manuscript drafting accounting for 60 Academic Hours (AHs).
Internship	Not applicable for this module.
Lab or Workshop	Hands-on workshops on LaTeX, BibTeX, and scientific visualization.
Field Work	Not applicable for this module.

Mode of assessment

Writing portfolio (60 %): a portfolio of four assessed pieces – a structured abstract, an introduction and related-work section of 4 to 6 pages, a data and method description with a reproducibility statement, and a one-page research proposal with timeline and resources – prepared in LaTeX and submitted with the version history.
Final presentation (40 %): an oral presentation of 20 to 25 minutes of the research proposal, followed by discussion.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Writing portfolio	60%
Final presentation	40%
Total	**100%**

Draft

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
Zobel, J. (2014). Writing for Computer Science, 3rd ed. Springer.	**Required**	
Booth, W. C., Colomb, G. G., Williams, J. M., et al. (2016). The Craft of Research, 4th ed. University of Chicago Press.	**Required**	
Glasman-Deal, H. (2020). Science Research Writing, 2nd ed. World Scientific.	Recommended	
Wohlin, C., Runeson, P., Hoest, M., et al. (2012). Experimentation in Software Engineering. Springer.	Recommended	

Material	Type	Material Locations
Higham, N. J. (2020). Handbook of Writing for the Mathematical Sciences, 3rd ed. SIAM.	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

Draft

MODULE DESCRIPTION

Module title **Master Thesis (incl. presentation / colloquium)**
Module code
Study program **Master of Computer Science**

Module coordinator/Lecturer

Type	Lecturer	Email	Office	Office hours (if any)
Module Coordinator	Dr. Tran Duc Khanh	khanh.td@vgu.edu.vn		By appointment
Lecturer	VGU Professors			By appointment
Tutorial	[TBD – to be appointed by the Faculty of Engineering]			By appointment
Lab	None			
Other	None			

Classification ☒ Compulsory ☐ Compulsory Elective ☐ Elective

Semester **4**

Mode of teaching ☒ Onsite ☐ Online ☐ Hybrid

Student workload

Credits	30	ECTS
Contact hours	30	AHs
Assignments and independent learning	870	AHs
Total Working hours	900	AHs

Frequency The module is offered each semester upon registration.

Prerequisites At least 60 ECTS credit points earned, in accordance with Article 9 of the Specific Examination Regulation. Command of the technical stack relevant to the chosen research topic and of scientific writing is expected.

Intended learning outcomes

On successful completion of this module the learner will be able to:

1. Analyze complex, open-ended problems in the field of Artificial Intelligence to identify original research gaps and formulate rigorous scientific hypotheses.
2. Design and implement a comprehensive research methodology or technical system that demonstrates an advanced mastery of computer science principles.
3. Evaluate experimental results or theoretical proofs using formal statistical methods, benchmarking, and critical comparison against the current state-of-the-art.
4. Critically synthesize research findings into a master's thesis that adheres to the highest standards of academic discourse and technical documentation.

Draft

5. Defend the research methodology and conclusions in an oral examination before a committee of academic experts.

Contents

Definition of the research topic with the supervising professor, and formal registration of the topic with the office of the study program.
Systematic review of the state of the art in the chosen sub-field, and the formulation of the research question and hypotheses.
Design of the research method: experimental framework, theoretical development or system architecture, with the validation strategy defined in advance.
Selection and preparation of datasets, computational environments and evaluation baselines.
Execution of the work: implementation, model development or formal analysis, with version control and documented experimental configurations.
Empirical or theoretical evaluation, including statistical treatment of results and comparison against the established baselines.
Critical discussion of the findings, their limitations, threats to validity and the implications for practice and for further research.
Consideration of the legal, ethical and societal implications of the work, including data protection and the disclosure of generative AI tools.
Preparation of the written thesis in English to international publication standards, with complete referencing and a reproducibility statement.
Preparation and delivery of the colloquium: presentation of the work and defence of the method and conclusions.

Learning activities

Individual supervision: regular scheduled meetings with the supervising professor throughout the six-month processing period (approximately 30 Academic Hours of contact).
Independent research: literature work, implementation, experimentation, evaluation and writing, accounting for approximately 870 Academic Hours.
Research colloquia: presentation of interim results to the supervising group where the Faculty offers them.
Use of the faculty laboratories, computing resources and the digital library.
Colloquium: a presentation of 25 to 35 minutes followed by discussion, with a total duration of 45 to 60 minutes, held no later than four weeks after submission of the thesis.
Thesis processing time: 6 months
Requirements for Starting the Master's Thesis
The Master's thesis can only be started if the following preconditions are fulfilled:

1. At least 60 CPs of course workload have been completed successfully.
2. The student has been able to define the proposed research topic and got approval from the supervising professor or lecturer. After the topic is approved, the candidate must register the Master's thesis with the office of the MCS study program. The topic and the date of issuance must be documented. The topic may only be returned once, and only within the first month of the processing period.

Colloquium
- Length of presentation: 25-35 minutes
- Total duration of the colloquium is at least 45 and no more than 60 minutes

Mode of assessment

Thesis report (80 %): the written master's thesis in English, assessed by two examiners in accordance with Article 23 of the General Examination Regulation and Article 10 of the Specific Examination Regulation.

Draft

Colloquium (20 %): a presentation of 25 to 35 minutes followed by discussion, total duration 45 to 60 minutes, generally open to the public.
Language of assessment: English. The repetition of examinations is governed by the General Examination Regulation. Assessment dates are announced in the semester examination schedule. Every graded assessment must be accompanied by the AI Disclosure Statement required by Article 8 of the Specific Examination Regulation. Graded oral components are conducted in accordance with Article 4a of the Specific Examination Regulation.

Grading policy

Assessment method	Percentage of total
Thesis report	80%
Colloquium	20%
Total	**100%**

Assessment dates are announced in the semester examination schedule and are not part of this module description.

Module materials

Material	Type	Material Locations
VGU/Faculty Guidelines for Master Theses	**Required**	
Specialized primary literature relevant to the chosen research topic	**Required**	
J. Zobel, *Writing for Computer Science*, Springer	Recommended	
W. Strunk, E.B. White, *The Elements of Style*	Recommended	
LaTeX templates for Master Theses (VGU standard)	Recommended	

Written/updated by
Prof. Dr.
Date: / /2026

Draft